



Token API And Signing JS SDK Integration

Third Party Provider API Specification Document



Document Control

Document Reference	Creation Date	Last Updated	Version
MTL/PRJ-NIT/RPT/03	30-June-21	22-Sept-21	1.1

Revision History

Version	Date	Contributors	Description of changes
1.0	30-Jun-21	Devesh Mishra	Release Version 1.0
1.1	02-Jul-22	Sekhar Cheedella Bhaskar Neeli	- Added UGHub Access Token Endpoint API - Added UgPass Access Token Endpoint API
1.2	13-Mar-24	Bhaskar Neeli	Updated JavaScript Sign Method Signature
1.5	29-Aug-26	Bhaskar Neeli	Updated JavaScript Sign Methods

Reviewed by

Name	Title
Ramamohan Reddy	MD, Digital Trust Technologies
Devesh Mishra	Project Manager, Digital Trust Technologies
Brenda Maniraguha	Project Manager, NITA-Uganda
James Lutalo	Technical Lead, NITA-Uganda

Distribution List

Name	Title
PIT	UGPASS Project Implementation Team

Table of Contents

1	<i>Introduction to Integrations with UgPass</i>	5
2	<i>Authentication Integration</i>	6
3	<i>Introduction to Integrations with UgPass</i>	6
4	<i>Service Provider Web Application integration with UgPass Authentication service</i>	6
4.1	Workflow diagram	6
4.2	Integration Steps:	7
4.2.1	Redirection to DAES for obtaining Authorization Code.....	7
4.2.1.1	Description:.....	7
4.2.1.2	Authorization Endpoint Request:.....	8
4.2.1.3	Request Parameters:.....	8
4.2.1.4	Request Token Header:.....	9
4.2.1.5	Request Token Payload:.....	9
4.2.1.6	Success Response:.....	9
4.2.1.7	Response Parameters:	9
4.2.1.8	Error Response:.....	10
4.2.1.9	List of Errors:.....	10
4.2.1.10	Sample Request:.....	10
4.2.1.11	Sample Success Response:.....	11
4.2.1.12	Sample Error Response:.....	11
4.2.2	Get Access Token with Authorization Code	11
4.2.2.1	Description:.....	11
4.2.2.2	Token Endpoint Request:.....	11
4.2.2.3	Request Parameters:.....	11
4.2.2.4	Client Assertion Token Header:	12
4.2.2.5	Client Assertion Token Payload:.....	12
4.2.2.6	Success Response:.....	12
4.2.2.7	Response Parameters:	13
4.2.2.8	ID Token Header & Payload:	13
4.2.2.9	ID Token Header:	13
4.2.2.10	ID Token Payload:	14
4.2.2.11	Error Response:	14
4.2.2.12	List of Errors:.....	14
4.2.2.13	Sample Curl Request:.....	15
4.2.2.14	Sample Success Response:.....	15
4.2.2.15	Sample Error Response:.....	16
4.2.3	Subscriber account linking.....	16
4.2.3.1	Description:.....	16
4.2.4	Check Subscriber's Level of Assurance	17
4.2.4.1	Description:.....	17
4.2.5	Logout user session from DAES.	17
4.2.5.1	Description:.....	17
4.2.5.2	Logout Request:	17
4.2.5.3	Request Parameters:.....	17
4.2.5.4	Success Response:.....	17
4.2.5.5	Error Response:.....	18

4.2.5.6	Sample Request:	18
4.2.5.7	Sample Response:	18
4.2.6	ID Token Verification with JWKS Url.....	18
4.2.6.1	JWKS Request:.....	18
4.2.6.2	Steps for validation of JWT token:	18
4.2.6.3	Sample Request:	19
4.2.6.4	Sample Success Response:	19
5	Agent deployment	20
6	Service Provider Web Application integration with UgPass Signature service.....	20
6.1	SDK Installation / Import	20
6.2	SDK Object Creation	20
6.3	On Success - Response Structure.....	21
6.4	On Failure - Response Structure	21
7	SDK Integration	21
8	SDK Functions	22
8.1	VerifySignatories	22
8.1.1	Description.....	22
8.1.2	Method Signature	22
8.1.3	Sample Request	22
8.1.4	Request Parameters	22
8.1.5	On Success - Response Structure.....	23
8.1.6	Response Parameters	23
8.1.7	On Failure - Response Structure	23
8.1.8	Error Codes and Error Messages.....	23
8.2	Preparation.....	24
8.2.1	Description.....	24
8.2.2	Method Signature	24
8.2.3	Sample Request	24
8.2.4	Request Parameters	24
8.2.5	On Success - Response Structure.....	24
8.2.6	Response Parameters	25
8.2.7	On Failure - Response Structure	25
8.2.8	Error Codes and Error Messages.....	25
8.2.9	Usage Example.....	26
8.3	DocumentSigning	27
8.3.1	Description.....	27
8.3.2	Method Signature	27
8.3.3	Sample Request	27
8.3.4	Request Parameters	28
8.3.5	On Success - Response Structure.....	28
8.3.6	Response Parameters	29
8.3.7	On Failure - Response Structure	29
8.3.8	Error Codes and Error Messages.....	29
8.3.9	Usage Example.....	29

Abbreviations

SP	Service provider
DSS	Digital signature services
ADES	Advanced electronic signatures
HSM	Hardware security module
CA	Certificate Authority
SAM	Signature Activation Module
RSS	Remote signature solution
QSCD	Qualified Signature Creation Device
CM	Crypto Module
eIDAS	Electronic Identification, Authentication and Trust Services
SMPC	Secure Multi-party Computation
QTSP	Qualified trust service provider
AdES	Advanced electronic signatures
QES	Qualified Electronic Signatures
drQSCD	Distributed remote Qualified Signature (or Seal) Creation Devices (QSCD)

Production Environment:

Base URL	https://api.ugpass.go.ug/idp/
UGHub Base URL	https://api.integration.go.ug/t/nita.go.ug/daes/1.0.0/idp/
UGHub Token URL	https://api.integration.go.ug/token

Staging Environment:

Base URL	https://stgapi.ugpass.go.ug/idp/
UGHub Base URL	https://api-uat.integration.go.ug/t/nita.go.ug/daes/1.0.0/idp/
UGHub Token URL	https://api-uat.integration.go.ug/token

1 Introduction to Integrations with UgPass

UgPass solution runs two services i.e., Authentication Service & Signing Service. This document describes the API specifications required by third party providers such as government MDAs & others to integrate to UgPass for the **authentication** service. The document further stipulates the Integration approach & API specifications for the **Signing** Service which will allow third party entities to invoke the signing service from their respective business application workflows.

2 Authentication Integration

Production Environment:

Base URL	https://api.ugpass.go.ug/idp/
----------	---

Staging Environment:

Base URL	https://stgapi.ugpass.go.ug/idp/
----------	---

3 Introduction to Integrations with UgPass

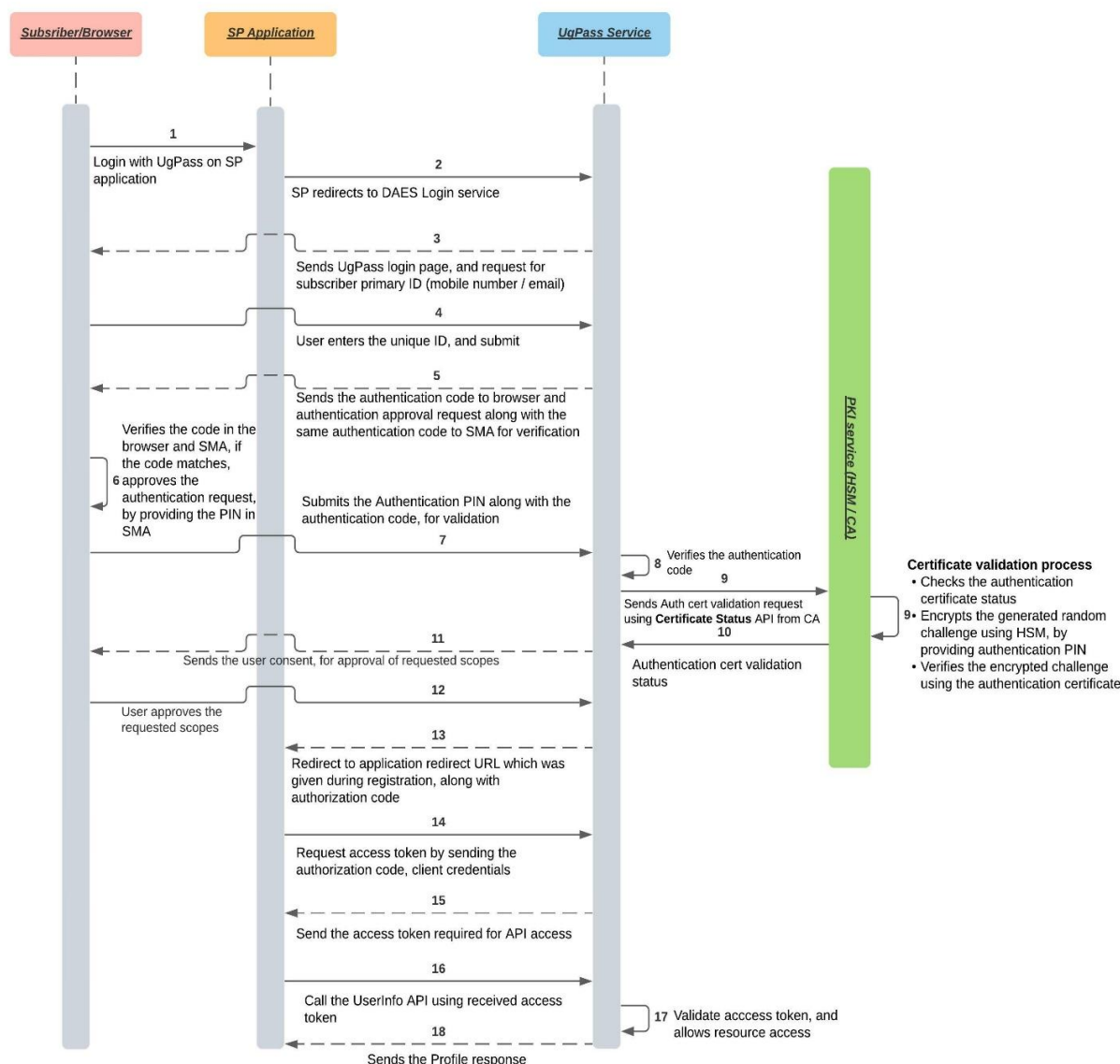
UgPass solution runs two services i.e., Authentication Service & Signing Service. This document describes the API specifications required by third party providers such as government MDAs & others to integrate to UgPass for the **authentication** service. The document further stipulates the Integration approach & API specifications for the **Signing** Service which will allow third party entities to invoke the signing service from their respective business application workflows.

4 Service Provider Web Application integration with UgPass Authentication service

This describes how to integrate service provider web application with UgPass. Through this integration, the Service Provider system will delegate the subscriber's digital authentication to DAES/UgPass which hosts the subscriber's account and will get access to account after subscriber has granted permissions & allowed their account details to be accessed.

4.1 Workflow diagram

Certificate Authentication



4.2 Integration Steps:

4.2.1 Redirection to DAES for obtaining Authorization Code.

4.2.1.1 Description:

Service Provider will redirect to DAES Login page and after successful subscriber authentication it will return Authorization Code to SP.

1. When subscriber clicked on “Login with DAES” in SP application. SP should redirect to the below Authorization Endpoint URL.
2. DAES Login page will be displayed and subscriber should submit unique identifier (Email/Mobile Number).
3. DAES will display authentication code in the browser and sends push notification to SMA installed in user’s device after subscriber identity is verified.
4. Subscriber when clicked on push notification, SMA will ask PIN or any other configured authentication method in his/her mobile device.
5. After successful device authentication SMA will display three random codes and subscriber

need to select the code displayed in the browser and submit the authentication PIN to approve authentication request.

6. After successful verification of code and authentication PIN, DAES will send the consent page for subscribers' approval to share requested information to SP.
7. If subscriber has approved, DAES will redirect to the SP Redirect URL containing the "code". This Redirect URL is the same as that SP provided during SP Registration.
8. This code can be used by the SP web application to obtain the access token and id_token.
9. If authentication was failed or any invalid parameters provided in request then error message is displayed in login page or redirected to SP with error and error description in query parameter.

4.2.1.2 Authorization Endpoint Request:

GET {Base URL}/authorization?client_id= {Client Id} &redirect_uri= {Redirect Uri}
&response_type=code& scope= {Space separated scopes} & state={State} &nonce={Nonce}
&request= {JWT}

4.2.1.3 Request Parameters:

Parameter Name	Parameter Type	Required	Description
client_id	String	True	Unique identifier of SP. It is generated when SP registered in DAES platform. This value will be provided by DAES.
redirect_uri	String	True	Indicates the url to return the subscriber to after authorization is complete. SP should provide this url during its registration.
response_type	String	True	Indicates that SP expects to receive an authorization code. This value should be "code".
scope	String	True	One or more scope values indicates which attributes of the subscriber's account SP application wish to access. This value will be provided by DAES. "openid","urn:idp:digitalid:profile" scopes are mandatory
state	String	True	A random string generated by SP, which can be stored and verified when the response is received from DAES. The primary reason for using this parameter is to mitigate XSRF attacks
nonce	String	True	A random string value that it is returned in the ID token. It is used to mitigate replay attacks.
request	JWT	True	Instead of passing all parameters in the URL query string, it is possible to move some or all of them to a JWT Token. JWT is signed by SP's Private key to further strengthen the

			validity and security by verifying the integrity and authenticity of origin of the Authorization Request.
--	--	--	---

4.2.1.4 Request Token Header:

Parameter Name	Parameter Type	Required	Description
alg	String	True	The signature algorithm used. We support RS256.
typ	String	True	The type of the token, which is JWT always.

4.2.1.5 Request Token Payload:

Parameter Name	Parameter Type	Required	Description
iss	String	True	It is same as client_id. Unique identifier of SP. It is generated when SP registered in DAES platform.
aud	String	True	It must be the value of DAES Authorization Server Issuer Url that will be provided by DAES.
iat	Integer	True	The time the JWT Token was created, represented in Unix time (seconds). It is used to evaluate the JWT Token lifetime.
exp	Integer	True	The time the JWT Token expires, represented in Unix time (seconds). It is used to evaluate the JWT Token lifetime.
nbf	Integer	False	Identifies the time before which the token must not be accepted for processing.
jti	String	False	The identifier of the token.
redirect_uri response_type scope state nonce	String	Recommended	The values in the JWT overrides the query URI values if the parameters are present in both query and in JWT.

4.2.1.6 Success Response:

HTTP/1.1 302 Found

Location: {SP Redirect Url}? code={Code}&state={State}

4.2.1.7 Response Parameters:

Parameter Name	Parameter Type	Description
Code	String	Authorization Code returned by DAES once


```
{
  "aud": "https://{BaseURL}/",
  "redirect_uri": "https:// client.example.com/oidc-login",
  "response_type": "code",
  "scope": "openid urn:idp:digitalid:profile",
  "nonce": "6fe230d7efb448b1a3ae6646a9517168",
  "state": "5e9bb6bbea8a40a482d7d4ce6006af72",
  "nbf": 1625852730,
  "exp": 1625856330
}
```

4.2.1.11 Sample Success Response:

HTTP/1.1 302 Found
 Location: https://client.example.com/oidc-login?code=AMGRKs6iAtNVlerwROdXGIIzNaL72wHcYulL&state=5e9bb6bbea8a40a482d7d4ce6006af72

4.2.1.12 Sample Error Response:

HTTP/1.1 302 Found
 Location: https://client.example.com/oidc-login?error=invalid_scope&error_description=The%20requested%20scope%20is%20invalid%20or%20not%20allowed%20for%20this%20client.&state=5e9bb6bbea8a40a482d7d4ce6006af72

4.2.2 Get Access Token with Authorization Code

4.2.2.1 Description:

Service Provider can call Token Endpoint url to get Access Token with authorization Code received in previous step. This Access Token can be used for accessing user information API and Signature API.

4.2.2.2 Token Endpoint Request:

POST /api/Authentication/token HTTP/1.1
 Host: {Base URL}
 Content-Type: application/x-www-form-urlencoded

grant_type = authorization_code & code = {Authorization Code} &redirect_uri = {Redirect Url} & client_id= {Client Id} & client_assertion_type = urn:ietf:params:oauth:client-assertion-type:jwt-bearer&client_assertion = {JWT Token}

4.2.2.3 Request Parameters:

Parameter Name	Parameter Type	Required	Description
Code	String	True	Authorization Code received in previous authorization request.
grant_type	String	True	SP should provide grant_type as authorization_code
redirect_uri	String	True	It should be same redirect_uri that was sent during authorization request.
client_id	String	True	Unique identifier of SP. It is generated when SP registered in DAES platform. This value will be provided by DAES.

client_assertion_type	String	True	Type of client_assertion. Its value will be urn:ietf:params:oauth:client-assertion-type:jwt-bearer
client_assertion	JWT	True	JWT Token created and signed by SP's Private Key to strengthen the validity and security by verifying the integrity and authenticity of origin of the Token Endpoint Request

4.2.2.4 Client Assertion Token Header:

Parameter Name	Parameter Type	Required	Description
Alg	String	True	The signature algorithm used. We support RS256.
Typ	String	True	Type of token, which is JWT.

4.2.2.5 Client Assertion Token Payload:

Parameter Name	Parameter Type	Required	Description
Iss	String	True	It is same as client_id. Unique identifier of SP. It is generated when SP registered in DAES platform.
Aud	String	True	It must be the value of DAES Authorization Server Token Endpoint Url.
iat	Integer	True	The time the JWT Token was created, represented in Unix time (seconds). It is used to evaluate the JWT Token lifetime.
Exp	Integer	True	The time the JWT Token expires, represented in Unix time (seconds). It is used to evaluate the JWT Token lifetime.
Sub	String	True	The subject of the token. This value must be the same as the client_id.
nbf	Integer	False	Identifies the time before which the token must not be accepted for processing.
jti	String	False	The identifier of the token.

4.2.2.6 Success Response:

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "access_token": {string}
  "token_type": {string},
  "expires_in": {Number},
```

```
"scopes": {string},
"id_token": {JWT}
}
```

4.2.2.7 Response Parameters:

Parameter Name	Description
access_token	UGPass Access Token used for API Access
token_type	Type of token. Value will be "Bearer".
expires_in	Access Token Expiry in seconds
Scopes	Scopes which are allowed for the access token
id_token	ID token is a JWT token that contains information about an authentication event and claims about the authenticated user. The JWT token is signed with DAES Private Key to strengthen the validity and security by verifying the integrity and authenticity of origin of the Token Endpoint Response.

4.2.2.8 ID Token Header & Payload:

```
{
  "alg": {string},
  "typ": {string},
  "kid": {string},
}

{
  "iat": {Number},
  "sub": {string},
  "aud": {string},
  "auth_time": {string},
  "at_hash": {string},
  "iss": {string},
  "nonce": {string},
  "exp": {Number},
  "daes_claims": {
    "suid": {string},
    "birthdate": {string},
    "name": {string},
    "phone": {string},
    "email": {string},
    "gender": {string},
    "id_document_type": {string},
    "id_document_number": {string},
    "loa": {string}
  }
}
```

4.2.2.9 ID Token Header:

Parameter Name	Description
Alg	The signature algorithm used. We support RS256.
Typ	The type of the token, which is JWT always
Kid	Identifies the public key used to verify the token. The corresponding public key can be found via the JWKS Url.

4.2.2.10 ID Token Payload:

Parameter Name	Description
Iss	It must be value of the DAES Authorization Server issuer Url
Aud	It is SP's client_id
Iat	The time the ID token was issued, represented in Unix time (seconds).
Exp	The time the ID token expires, represented in Unix time (seconds).
Sub	The subject of the token. A unique identifier for the user.
Nonce	A value that is sent in authorization endpoint is returned in the ID token. It is used to mitigate replay attacks.
at_hash	Access Token hash value. For instance, if the alg is RS256, hash the access_token value with SHA-256, then take the left-most 128 bits and base64url encode them. This value is used to verify the integrity of access_token
auth_time	The time the end user was authenticated, represented in Unix time (seconds).
Suid birthdate name phone email gender id_document_type id_document_number loa	For Description, please refer to UserInfo Endpoint Response.

4.2.2.11 Error Response:

HTTP/1.1 401 Unauthorized
Content-Type: application/json

```
{
  "error": {String}
  "error_description": {String}
}
```

4.2.2.12 List of Errors:

Error Code	Error Description
Invalid_credentials	SP credentials sent in authorization header are not valid
Invalid_authentication_scheme	The authentication scheme specified in the Authorization header is not valid or not supported.
Invalid_client	SP client application was not valid
unsupported_grant_type	Grant type in the request body is not valid or not supported by DAES

```
curl -L -X POST "{Base URL}/api/Authentication/token" ^
-H "Content-Type: application/x-www-form-urlencoded" ^
-d "code=ttA3XuIfF8ALvby0nLn6A8z0RBhse3ZFqioy" ^
-d "grant_type=authorization_code" ^
-d "redirect_uri=https://client.example.com/oidc-login" ^
-d "client_id=1gl0heedStC3XvpMilyIzPaVof64PqvJbWS4vUlyXIHeBQM6" ^
-d "client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer" ^
-d "client_assertion=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjE5OTk1eGivzz.....7DItBJdu2m1ThvAXb4ooJR8ieRKLC kZw"
```

```
{
  "alg": "RS256",
  "typ": "JWT"
}
```

```
{
  "iat": "1625852855",
  "jti": "b3a3b06f-d9b6-407d-9f8a-669db483a4e6",
  "iss": "1gl0heedStC3XvpMllylZPaVof64PqvJbWS4vUlyXIHeBQM6",
  "aud": "https://{BaseURL}/api/Authentication/token",
  "sub": "1gl0heedStC3XvpMllylZPaVof64PqvJbWS4vUlyXIHeBQM6",
  "nbf": 1625852855,
  "exp": 1625856455
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "access_token": "mwACT8EldqdTlbRn0gZCCNfcvmUI+tpNc/sUvDul94CYcZ9najY0jF8PtpmoC78e5lQ+h9POckZ4m7KSw3h8PMJY9Le2xPGuLpXUp+RD9vYwKmA7bzyOpeAXJ7AiJfwlFBcG0/vwvP/A+PfiJuv8rcHLOe tvQ+sARSGmHpoPSYEeFdFeR9sf4gLOCchUrVTMIk9hO54Px37BCkldPsbxTjt6YzxiXJYMODkUGiuFJyR1L89Q1GBCVojGzSNZbr5K",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scopes": "openid urn:idp:digitalid:profile",
  "id_token": "eyJhbGciOiJIUzI1dkioNiz.....tQ0s9BWmtu83v5_fwXFLhs8-Oxr3cgES-lIi8SxITI27g"
}
```

{

```

"alg": "RS256",
"kid": "256e1063-87e3-403c-a289-d866d71bbd70",
"typ": "JWT"
}

Payload:
{
  "iss": "{BaseURL}",
  "sub": "c9283b55-6f62-40b2-acf8-bf7482648ff7",
  "aud": "1gl0heedStC3XvpMllylzPaVof64PqvJbWS4vUlyXIHeBQM6",
  "exp": 1625853513,
  "iat": 1625852913,
  "nonce": "6fe230d7efb448b1a3ae6646a9517168",
  "auth_time": 1625852913,
  "at_hash": "_OJpoAnzzONhS_W95OoK7Q",
  "daes_claims": {
    "suid": "c9283b55-6f62-40b2-acf8-bf7482648ff7",
    "birthdate": "4/10/1979 12:00:00 AM",
    "name": "Test User",
    "phone": "+256XXXXXXXXX",
    "email": "XXXXXXXXXX@gmail.com",
    "gender": "FEMALE",
    "id_document_type": "1",
    "id_document_number": "XXXXXXXXXX",
    "loa": "LOA1"
  }
}

```

4.2.2.15 Sample Error Response:

```

HTTP 401 Unauthorized
Content-Type: application/json
{
  "error": "invalid_client",
  "error_description": "Client authentication failed"
}

```

4.2.3 Subscriber account linking

4.2.3.1 Description:

Service Provider after receiving subscriber information can identify the subscriber and link the subscriber DAES account with SP account and login into application. Every subscriber in DAES is uniquely identified by SUID.

Case 1: Subscriber who logged into DAES is already registered with Service provider application.

1. SP application can link the subscriber account automatically by considering the unique parameters like NIN, Passport Number, Email, Mobile Number.
2. SP can link subscriber account manually by showing SP login page and after successful authentication SP can link the SUID, NIN, to the subscriber SP account.
3. "SUID" attribute returned by DAES can be used for further user linking purposes.

Case 2: Subscriber who logged into DAES is not registered with Service provider application.

1. SP should allow the subscriber to register in the application with the information received from DAES.
2. "SUID" attribute returned by DAES can be used for further user linking purposes.

4.2.4 Check Subscriber's Level of Assurance

4.2.4.1 Description:

Service Provider can get the subscriber LOA in the ID Token Claims. SP should handle the flow based on the subscriber's LOA. LOA also defines the method by which subscriber was Onboarded into DAES. There are three Level of Assurance.

LOA1: In this case Subscriber is onboarded into DAES with UNID (Uganda National ID Card) and NIN is returned in ID Token Claims.

LOA2: In this case Subscriber is onboarded into DAES with Passport and Passport number is returned in the ID Token Claims.

LOA3: In this case Subscriber is onboarded into DAES with E-Passport or NIN (NIN verified with NIRA API). NIN or E-Passport Number is returned in the ID Token Claims.

4.2.5 Logout user session from DAES.

4.2.5.1 Description:

After successful DAES Authentication in one SP web application it will not ask authentication in another SP web application as DAES supports SSO in same browser. When logout is clicked in SP, it should redirect to DAES Logout url by specifying SP logout url in the query parameter. DAES will clear the subscriber session and it will redirect to SP application logout url received in query parameter.

4.2.5.2 Logout Request:

GET {BaseURL}/OIDClogout? id_token_hint = {id_token} & post_logout_redirect_uri = {SP Logout Url} & state = {state}

4.2.5.3 Request Parameters:

Parameter Name	Parameter Type	Required	Description
id_token_hint	JWT	True	It was same as id_token that DAES sent in Token Request. A valid ID token with a subject that matches the current session.
post_logout_redirect_uri	String	True	Location to redirect to after the logout is performed. It must match the value preregistered in DAES during client registration.
State	String	True	A random string generated by SP, which can be stored and verified when the response is received from DAES. The primary reason for using this parameter is to mitigate XSRF attacks

4.2.5.4 Success Response:

HTTP 302 Found

Location: {SP Logout Url}?state={state}

4.2.5.5 Error Response:

- If the ID token passed via id_token_hint is invalid then DAES will send error page.
- If the DAES subscriber session has expired (or doesn't exist), a logout request simply redirects to the post_logout_redirect_uri.
- If the ID token is valid, but expired, and the subject matches the current DAES subscriber session, a logout request logs the user out and redirects the browser to the post_logout_redirect_uri.

4.2.5.6 Sample Request:

```
GET {BaseURL}/OIDClogout?
id_token_hint=eyJhbGciOiJSUzI1dkioNiz.....tQ0s9BWmtu83v5_fwXFLhs8-Oxr3cgES-Ili8SxITI27g &
post_logout_redirect_uri=https://client.example.com/oidc-logout&
state=967e0f511bf24a44968f0249e4d48517
```

4.2.5.7 Sample Response:

```
HTTP/1.1 302 Found
Location: https://client.example.com/oidc-logout?state=967e0f511bf24a44968f0249e4d48517
```

4.2.6 ID Token Verification with JWKS Url

Description:

ID token received in token endpoint response is JWT token which are signed with DAES private key.

4.2.6.1 JWKS Request:

GET http://{BaseURL}/api/Jwks/jwksuri

4.2.6.2 Steps for validation of JWT token:

- Need to decode the token, which is in JWT format.
- The JSON Web Key Set (JWKS) need to be retrieved from DAES Authorization Server and cached in application which is required for Signature verification.
- Verify the token's signature by matching the key that was used to sign in with one of the keys retrieved from DAES Authorization Server's JWK endpoint. Specifically, each public key is identified by a kid attribute, which corresponds with the kid claim in the token header.
- If the kid claim doesn't match, it's possible that the signing keys have changed. Try retrieving the keys again from server.
- The iss(issuer) claim should match with DAES Authorization Server Issuer Url.
- The aud(audience) claim should match the Client ID that is used to request the ID Token, which is the Client ID of the SP.
- The iat(issued at time) claim in id_token indicates when this token was issued, expressed in Unix time.
- The exp (expiry time) claim in id_token is the time at which this token will expire., expressed in Unix time. Expired token is not valid.
- The nonce claim value in id_token should match with nonce value that was passed when requested the ID token.

- The at_hash claim value in id_token should match with the value obtained by performing the SHA 256 hash on access_token value and then taking the left-most 128 bits and base64url encoded value.

4.2.6.3 Sample Request:

```
GET https://{BaseURL}/api/Jwks/jwksuri
```

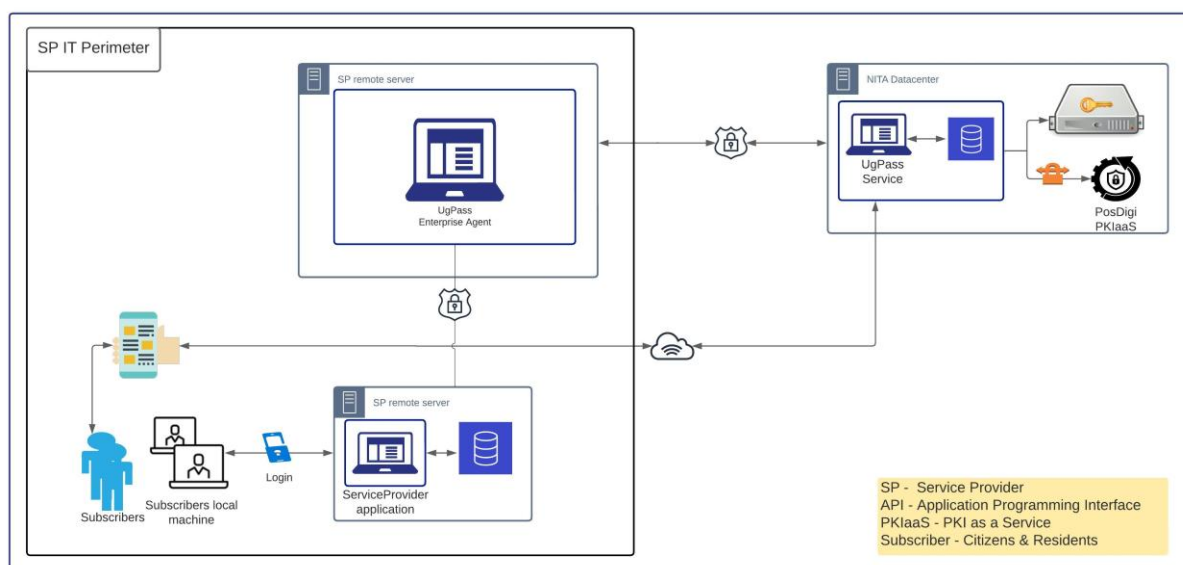
4.2.6.4 Sample Success Response:

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "keys": [
    {
      "kty": "RSA",
      "kid": "SmAZ-WscXAfIF-EBavrsv6DxLfd5NJS_Wt9rmyD4lo8",
      "use": "sig",
      "alg": "RS256",
      "n": "heQQlppIRWBqGdHV8dRc-
cka6kyJT36tUd5sUZXLd8pKnE00Bpo3qryNNN_liSu_wnVwTDqgNmGudkpFkRg1TqjqhdlC18fcasN-
wM2rxkxH3OqsRD48trQynHOWDxYJdQcwi8cmPuowfFOCAfH23NgTEhHswKyr9PUOpFRHI_Lbuu14uv_YD
aBJ2wn2SQ77inOuNCOZOofsm7QdfZRIxbNXMZFM7Z6yLnD_PhNlcFOzgpGAXfCM2jBdPIP1po3wtVd7O2
zMYeIGUVIEodARxvWyyfVycqSqcGsgyVFJYvl5Bac_V3LRvoQXgPXuJo_bLTDWwBQOqdLUvnwAxjnHQ",
      "e": "AQAB"
    },
    {
      "kty": "RSA",
      "kid": "256e1063-87e3-403c-a289-d866d71bbd70",
      "use": "sig",
      "alg": "RS256",
      "n": "vpWMf11Ui0d9jY5veSoFAkuyCjqM3HvY6a49h-LY88nQ3jiDHlVwwiNfewJ8s7gzKtcv81CINw2-
ElPSW9yZVRtfdUgSu-72u6urAjGZuKtmQovlrAJpC2RjbnEBNWlvgCuE2yVeZ1A7giPDzT_jxfu-
qUAZJpb5PUH-imUsL5PtFFuMTRcNSf_Jf6unH4zMe9KtKz29lGRkX25ESKqVgoulO91izDKiGu72k-W-
T4bglklaUmYISAJsfO-
PfpvVTKXKav2x_xu460oNHEDjDdzBx6KnzifvmF26FTN0r458kABz8lXLeoM_uDrgFGB2dbxfuRwx3z1MnuCn
xp8MJQ",
      "e": "AQAAAAE"
    }
  ]
}
```

5 Agent deployment



Production Environment:

Signing Base URL	{{Defined by the Entity}}
------------------	---------------------------

Staging Environment:

Signing Base URL	{{Defined by the Entity}}
------------------	---------------------------

6 Service Provider Web Application integration with UgPass Signature service

6.1 SDK Installation / Import

```
import { EGPPPreparationSDK } from "../preparation-sdk/document-preparation-sdk.js";
```

6.2 SDK Object Creation

```
const egpSdk = new EGPPPreparationSDK();

const initResponse = await egpSdk.initialize({
  baseUrl: "https://staging.digitaltrusttech.com/native-dss/api/egp/",
  accessToken: accessTokenFromEgp,
  timeoutMs: 30000, #optional
});

if (initResponse.status !== "SUCCESS") {
```

```
throw new Error(initResponse.errorMessage);
}
```

6.3 On Success - Response Structure

```
{
  "status": "SUCCESS",
  "errorCode": null,
  "errorMessage": null,
  "result": {
    "initialized": true
  }
}
```

6.4 On Failure - Response Structure

```
{
  "status": "ERROR",
  "errorCode": "INVALID_CONFIGURATION",
  "errorMessage": "accessToken is required.",
  "result": null
}
```

Security: In production, access tokens should be short-lived and handled according to the EGP authentication architecture. Do not place long-lived server credentials or Agent Signing API secrets in browser code.

7 SDK Integration

The application integration consists of three business operations: verify signatories, prepare document, and sign document.

Operation	Application provides	SDK returns
Verify Signatories	documentIds	transactionId
Preparation	PDF + transactionId	QR-embedded PDF + signing configuration
Signature	Preparation result from preparation operation	Final signed PDF

Integration boundary: The application should not construct requestedActions, signer type, organization, eSeal authorization, annotation coordinates, or workflow IDs. Those are internal to the preparation/signing workflow and are represented in the signing configuration returned by the SDK.

8 SDK Functions

The SDK exposes three application-facing business functions: `verifySignatories()`, `prepareDocument()`, and `signDocument()`. All functions return Promise-based response envelopes.

8.1 VerifySignatories

8.1.1 Description

The `verifySignatories()` function verifies the signer information associated with the supplied document IDs before document preparation.

8.1.2 Method Signature

```
const response = await egpSdk.verifySignatories({
  documentIds,
  onComplete
});
```

NOTE: `onComplete` is optional. It receives the same response envelope returned by the Promise, including `result.transactionId` on success.

8.1.3 Sample Request

```
await egpSdk.verifySignatories({
  documentIds: ["document-id-1", "document-id-2"],
  onComplete: (response) => {
    if (response.status === "SUCCESS") {
      console.log(response.result.transactionId);
    } else {
      console.error(response.errorCode, response.errorMessage);
    }
  }
});
```

8.1.4 Request Parameters

Parameter	Type	Required	Description
<code>documentIds</code>	<code>array[string]</code>	Yes	List of document IDs for which signer and organization information needs to be verified.
<code>onComplete</code>	<code>function</code>	No	Optional callback invoked when verification completes. It receives the normal verification response envelope.

8.1.5 On Success - Response Structure

```
{ "status": "SUCCESS",
  "errorCode": null,
  "errorMessage": null,
  "result": { "transactionId": "generated-uuid" }
}
```

8.1.6 Response Parameters

Parameter	Type	Description
status	string	Response status. SUCCESS indicates that signer verification completed successfully
errorCode	string null	Null on success.
errorMessage	string null	Null on success.
result.transactionId	string	Transaction ID generated after successful verification. This ID is subsequently passed to prepareDocument()

8.1.7 On Failure - Response Structure

```
{ "status": "ERROR",
  "errorCode": "SIGNER_INFO_UNAVAILABLE",
  "errorMessage": "One or more document IDs could not be verified.",
  "result": { "invalidDocumentIds": [ "document-id-2" ] }
}
```

8.1.8 Error Codes and Error Messages

Error Code	Example Message	Typical Cause
INVALID_REQUEST	Invalid preparation request.	A required request field is missing or malformed.
INVALID_BASE64_PDF	The document is not valid Base64 PDF data.	The supplied document cannot be decoded as a PDF.
EMPTY_PDF	The PDF document is empty.	The decoded PDF contains no usable document content.
INVALID_SIGNER_LIST	Document ID list is required.	No document IDs were supplied.
SIGNER_INFO_UNAVAILABLE	One or more document IDs could not be verified.	One or more supplied document IDs did not return valid signer/organization details.
ORGANIZATION_UNAVAILABLE	No applicable organization is available.	Required organization information cannot be resolved.
ESEAL_NOT_AUTHORIZED	Signer is not authorized for eSeal.	The selected organization does not permit the requested eSeal action.
REQUIRED_ANNOTATIONS_MISSING	Required annotations are incomplete.	One or more required placements were not configured.
PREPARATION_API_FAILED	Preparation service request failed.	Backend or network failure while persisting preparation.
API_TIMEOUT	The API request timed out.	Backend response exceeded the SDK timeout.
INVALID_API_RESPONSE	The API response is invalid.	Backend returned malformed or unexpected data.

8.2 Preparation

8.2.1 Description

The preparation function accepts the source PDF and the transactionId returned by verifySignatories(). The SDK retrieves the validated signer and organization details from its transaction cache. The user places the required annotations in the preparation UI.

8.2.2 Method Signature

```
const response = await egpSdk.prepareDocument({ request });
```

8.2.3 Sample Request

```
const preparationResponse = await egpSdk.prepareDocument({
  request: {
    document: pdfBase64,
    fileName: "agreement.pdf",
    transactionId: verificationResponse.result.transactionId
  }
});
```

8.2.4 Request Parameters

Parameter	Type	Required	Description
request	object	Yes	Container for the preparation request data.
request.document	string	Yes	The source PDF encoded as a Base64 string. The value should represent the original PDF supplied for preparation.
request.transactionId	string	Yes	Transaction ID returned by verifySignatories().
request.fileName	string	No	Original PDF filename, for example agreement.pdf. Recommended because Base64 does not reliably include the original filename.

8.2.5 On Success - Response Structure

```
{
  status: "SUCCESS",
  errorCode: null,
  errorMessage: null,
  result: {
    requestId: "550e8400-e29b-41d4-a716-446655440000",
    preparedDocument: {
      id: "7b1f2e0a-5c9d-4f28-9c17-22e9d2a8f101",
      data: "<PREPARED_PDF_BASE64>"
    }
  }
}
```

```

    },
    signingConfiguration: {
      signers: [
        // Resolved signer configuration
      ],
      annotations: [
        // Prepared annotation information
      ]
    }
  }
}

```

8.2.6 Response Parameters

Parameter	Type	Description
status	string	Response status. SUCCESS indicates that preparation completed successfully.
errorCode	string null	Null on success.
errorMessage	string null	Null on success.
result.requestId	string	Original request identifier returned for correlation.
result.preparedDocument.id	string	Identifier of the prepared document.
result.preparedDocument.data	string	Prepared PDF encoded as Base64. This document already contains applicable INITIAL and QR_CODE embeddings.
result.signingConfiguration	object	Resolved configuration used by the signing operation, including signer information, requested actions, organization information, authorization state, and annotations.

8.2.7 On Failure - Response Structure

```

{
  "status": "ERROR",
  "errorCode": "INVALID_REQUEST",
  "errorMessage": "transactionId is unknown or has expired. Call verifySignatories() again.",
  "result": null
}

```

8.2.8 Error Codes and Error Messages

Error Code	Example Message	Typical Cause
INVALID_REQUEST	Invalid preparation request.	A required request field is missing or malformed.
INVALID_BASE64_PDF	The document is not valid Base64 PDF data.	The supplied document cannot be decoded as a PDF.
EMPTY_PDF	The PDF document is empty.	The decoded PDF contains no usable document content.
INVALID_SIGNER_LIST	Document ID list is required.	No document IDs were supplied.
SIGNER_INFO_UNAVAILABLE	One or more document IDs could not be verified.	One or more supplied document IDs did not return valid signer/organization details.

ORGANIZATION_UNAVAILABLE	No applicable organization is available.	Required organization information cannot be resolved.
ESEAL_NOT_AUTHORIZED	Signer is not authorized for eSeal.	The selected organization does not permit the requested eSeal action.
REQUIRED_ANNOTATIONS_MISSING	Required annotations are incomplete.	One or more required placements were not configured.
PREPARATION_API_FAILED	Preparation service request failed.	Backend or network failure while persisting preparation.
API_TIMEOUT	The API request timed out.	Backend response exceeded the SDK timeout.
INVALID_API_RESPONSE	The API response is invalid.	Backend returned malformed or unexpected data.

8.2.9 Usage Example

```
const verificationResponse = await egpSdk.verifySignatories({
  documentIds: ["document-id-1", "document-id-2"]
});

if (verificationResponse.status === "ERROR") {
  console.error(verificationResponse.result.invalidDocumentIds);
  return;
}

const preparationResponse = await egpSdk.prepareDocument({
  request: {
    document: pdfBase64,
    fileName: "agreement.pdf",
    transactionId: verificationResponse.result.transactionId
  }
});

if (preparationResponse.status === "ERROR") {
  console.error(
    preparationResponse.errorCode,
    preparationResponse.errorMessage
  );
  return;
}
```

```
const preparationResult = preparationResponse.result;
```

8.3 DocumentSigning

8.3.1 Description

The SDK validates the supplied SUID against `prepareDocumentRequest.signingConfiguration.signers`. If it matches, the SDK opens the signing workspace in a new tab and displays only that signer's configured annotations. The signing API is not called at this stage.

The user must click **Confirm Signing** in the signing workspace before the SDK calls the signing API. If the SUID does not match, the SDK returns an error, does not open the signing workspace, and does not call the signing API.

8.3.2 Method Signature

```
const response = await egpSdk.signDocument({
  prepareDocumentRequest,
  suid,
  accessToken,
  onComplete
})
```

NOTE: `onComplete` is optional. It receives the same response envelope returned by the Promise, including `response.result.finalDocument.data` on success.

8.3.3 Sample Request

```
const signingResponse = await egpSdk.signDocument({
  prepareDocumentRequest: {
    requestId: "550e8400-e29b-41d4-a716-446655440000",
    preparedDocument: {
      id: "7b1f2e0a-5c9d-4f28-9c17-22e9d2a8f101",
      data: "<PREPARED_PDF_BASE64>"
    },
  },
  signingConfiguration: {
    signers: [
      // Resolved signer configuration
    ],
    annotations: [
      // Sign/eSeal annotation information
    ]
  },
  suid: "fa381059-232e-470a-a3b3-741ae8a01153",
  accessToken: accessToken,
```

```
onComplete: (response) => {
    console.log("Signed document response:", response);
}
});
```

8.3.4 Request Parameters

Parameter	Type	Required	Description
prepareDocumentRequest	object	Yes	The complete result from the successful prepareDocument operation.
prepareDocumentRequest.requestId	string	Yes	Request identifier that correlates signing with the preparation transaction.
prepareDocumentRequest.preparedDocument	object	Yes	Prepared PDF metadata and Base64 document returned from preparation.
prepareDocumentRequest.preparedDocument.id	string	Yes	Prepared document identifier.
prepareDocumentRequest.preparedDocument.data	string	Yes	Prepared PDF Base64 data.
prepareDocumentRequest.signingConfiguration	object	Yes	Configuration produced by the preparation phase. The application should pass this object unchanged.
suid	string	Yes	SUID of the signer who will sign. It must match a signer in prepareDocumentRequest.signingConfiguration.signers.
onComplete	function	No	Callback invoked after signing completes, with the same signing response returned by signDocument().
accessToken	string	No	Optional token override. The SDK sends it in the UgPassAuthorization: Bearer <token> HTTP header. It is not included in the multipart request body. If omitted, the SDK uses the token provided during initialize().

Pass-through rule: The application should pass the complete preparation result to signDocument(). It should not reconstruct or modify the prepared document, signer configuration, or annotation configuration between phases unless the SDK contract explicitly permits it.

Signer selection: The application provides suid. The SDK validates it against prepareDocumentRequest.signingConfiguration.signers, opens the signing workspace for the matching signer, and displays only that signer's annotations. The SDK sends only that signer's signing annotations to the signing API after the user clicks **Confirm Signing**. An invalid SUID prevents the signing workspace and signing API request.

8.3.5 On Success - Response Structure

```
{
  status: "SUCCESS",
  errorCode: null,
  errorMessage: null,
  result: {
    requestId: "550e8400-e29b-41d4-a716-446655440000",
    finalDocument: {
      id: "DOC-FINAL-0001",
      data: "<FINAL_SIGNED_PDF_BASE64>"
    }
  }
}
```

8.3.6 Response Parameters

Parameter	Type	Description
status	string	Response status. SUCCESS indicates that the signing workflow completed successfully.
errorCode	string null	Null on success.
errorMessage	string null	Null on success.
result.requestId	string	Original request identifier returned for correlation.
result.finalDocument.id	string	Identifier of the final signed document.
result.finalDocument.data	string	Final signed PDF encoded as Base64.

8.3.7 On Failure - Response Structure

```
{
  "status": "ERROR",
  "errorCode": "SIGNING_API_FAILED",
  "errorMessage": "The signing service request failed.",
  "result": null
}
```

8.3.8 Error Codes and Error Messages

Error Code	Example Message	Typical Cause
INVALID_REQUEST	Invalid signing request.	Preparation result is missing or malformed.
PREPARATION_NOT_READY	Preparation result is not ready for signing.	Preparation status/configuration is incomplete.
AUTHENTICATION_FAILED	Signer authentication failed.	Required signer authentication context is invalid or expired.
SESSION_EXPIRED	The signer session has expired.	The signing session or authentication context is no longer valid.
SIGNING_CANCELLED	The signer cancelled the signing operation.	User cancelled before signing confirmation.
SIGNING_API_FAILED	The signing service request failed.	Backend or Agent Signing API failure.
ESEAL_FAILED	The eSeal operation failed.	eSeal execution failed after authorization.
INSUFFICIENT_CREDITS	Insufficient credits for the requested action.	Applicable signer/account balance is insufficient.
DOCUMENT_REVISION_CONFLICT	The submitted document revision is not the latest revision.	Another workflow operation changed the document before signing.
API_TIMEOUT	The signing API request timed out.	Backend did not respond within the configured timeout.
INVALID_API_RESPONSE	The signing API returned an invalid response.	Backend response is malformed or incompatible.
INVALID_CONFIGURATION	A signer SUID is required. / The supplied SUID does not match a signer for this document.	suid is missing or does not match the prepared signer configuration.

8.3.9 Usage Example

```
// preparationResult is the result returned by prepareDocument().
```

```
await egpSdk.signDocument({
  prepareDocumentRequest: preparationResult,
  suid: selectedSignerSuid,
  accessToken: accessToken,
  onComplete: (response) => {
    if (response.status === "ERROR") {
      console.error(response.errorCode, response.errorMessage);
      return;
    }

    const finalPdfBase64 = response.result.finalDocument.data;
    saveFinalDocument(finalPdfBase64);
  }
});
```

Note:The external application normally provides the SUID directly. The SDK validates the SUID and opens the signing workspace with only the matching signer's annotations. The signing API is called only when the user clicks Confirm Signing. After successful signing, the signed PDF is returned through onComplete. If the SUID is omitted, the SDK displays the SUID popup for local testing.